Program Generation via Reinforcement Learning with Enumeration Warm-Start

Yiding Song

April 21, 2026

Problem Statement

We are interested in the following problem. Suppose we have access to:

- A set F of functions $f_1, f_2, \ldots, f_n$ with types $\tau_1 \to \upsilon_1, \tau_2 \to \upsilon_2, \ldots, \tau_n \to \upsilon_n$, where we can query the function implementation (i.e. $f_i(x)$ can be computed for arbitrary well-typed inputs x) but not necessarily the function description (e.g. we do not know the mathematical or programmatic form of the functions).
- A reward function $R : \text{program} \to \mathbb{R}$ that maps a program to a real-valued reward, where a program is a well-typed composition of functions from F (e.g. this can reward a function for desirable properties such as being non-constant).

How do we synthesise a large number of programs that are diverse and achieve high rewards?

This document describes a two-phase program synthesis solution to the problem: bottom-up enumeration followed by reinforcement learning.

Overview

The pipeline has two phases:

1. **Bottom-up enumeration** — exhaust all programs up to a certain size, quotienting out semantic duplicates (as measured by function behaviour on a test suite), creating a corpus of behaviourally diverse programs.
2. **RL exploration** — a neural policy learns from the corpus (behavioural cloning warm-start), then generates new programs via policy gradient, using a priority queue buffer of the best programs found so far (Abolafia et al., 2018).

1 The Program Grammar

Programs are expressions in a simply-typed lambda calculus extended with conditionals and a fixed set of primitive functions. The expression language is:

$$e \ ::= \ c \ \mid \ x \ \mid \ f(e_1, \ldots, e_k) \ \mid \ \lambda(x_1, \ldots, x_k).\, e \ \mid \ \text{if } e_1 \text{ then } e_2 \text{ else } e_3$$

where c ranges over literals (integers, booleans, empty lists), x over variables, and f over the primitive functions in F.

1.1 Types

The base types are int and bool. From these, list types list[σ] and function types $\sigma \to \tau$ are formed. In the current instantiation the concrete types in use are:

$$\text{int}, \quad \text{bool}, \quad \text{list[int]}, \quad \text{list[bool]}, \quad \text{list[list[int]]}$$

All programs synthesised by the pipeline have type list[int] $\to$ list[int].

1.2 Polymorphic Functions

Some primitives in F are **polymorphic**: their types contain type variables that can be instantiated to any concrete type. For example:

$$\text{map} : (T_1 \to T_2) \to \text{list}[T_1] \to \text{list}[T_2]$$

Here T_1 and T_2 are type variables. A **type substitution** θ maps each type variable to a concrete type; applying θ to the signature of map yields a monomorphic instantiation. For instance, $\theta = \{T_1 \mapsto \text{int}, T_2 \mapsto \text{bool}\}$ gives:

$$\text{map}_\theta : (\text{int} \to \text{bool}) \to \text{list[int]} \to \text{list[bool]}$$

The set of valid substitutions for a function is determined by the concrete types in use. Each instantiation is treated as a separate monomorphic function during enumeration.

1.3 Higher-Order Functions

A function is **higher-order** if one or more of its arguments has a function type. `map` is higher-order: its first argument is a function $T_1 \to T_2$. Such arguments cannot be looked up directly in the program bank (which stores complete expressions, not open function values); instead they must be synthesised as lambda expressions, as described in the Higher-Order Applications section below.

2 Part 1: Bottom-Up Enumeration

2.1 Size Metric

Every AST node e is assigned an integer size $|e|$ inductively:

Node	Size
Literal, variable, empty list	1
Application $(f\ a_1\ \cdots\ a_k)$	$1 + \lvert a_1\rvert + \cdots + \lvert a_k\rvert$
Lambda $(\lambda\ (p_1\ \cdots\ p_k)\ e)$	$1 + \lvert e\rvert$
Conditional (if $c\ t\ e$)	$1 + \lvert c\rvert + \lvert t\rvert + \lvert e\rvert$

Enumeration proceeds by increasing size: programs of size s are built exclusively from sub-expressions of size less than s.

2.2 Observational Equivalence and the Program Bank

We remove program duplicates by comparing their behaviour on a test suite. Fix a test suite $T = \{t_1, \ldots, t_m\}$ of m concrete inputs. The *fingerprint* of a closed program p of type $\tau \to \upsilon$ is:

$$\Phi(p) = \big(p(t_1),\ \ldots,\ p(t_m)\big) \in (\upsilon \cup \{\dagger\})^m$$

where † records a failed evaluation (runtime error or type error). Two programs $p, q : \tau \to \upsilon$ are observationally equivalent if $\Phi(p) = \Phi(q)$.

Open terms and lambda arguments. However, not all programs are closed. To enumerate an expression like map $(\lambda y.\ e)\ x$, the enumerator must synthesise candidate bodies e. These bodies are *open terms*: they may reference y (the lambda parameter) freely, alongside the outer x. Storing them in the bank allows the same body to be reused across different higher-order functions, avoiding redundant work.

In general, for an open term e with free variables $y_1 : \sigma_1, \ldots, y_k : \sigma_k$ (beyond the primary input x), the fingerprint is computed over $T \times V_1 \times \cdots \times V_k$, where each V_i is a finite set of probe values for type σ_i:

$$\Phi(e) = \big(e(t,\ v_1, \ldots, v_k)\big)_{(t,\, v_1, \ldots, v_k)\, \in\, T \times V_1 \times \cdots \times V_k}$$

Two open terms are considered observationally equivalent iff they agree on this full Cartesian product.

The **program bank** $\mathcal{B}$ maps each pair (τ, s) to a set of programs, containing exactly one representative per observational equivalence class:

$$\mathcal{B}[\tau][s] = \big\{ [p]_\sim \ \big|\ \vdash p : \tau,\ |p| = s \big\}$$

Default test suite $(m = 10)$:

```
[], [0], [3,1,2], [1,1,1,1], [5,4,3,2,1],
[1,2,3,4,5,6,7,8], [10,-3,7,7,0], [2,8,3,8,2,3],
[0,1,0,1,0], [42]
```

2.3 Base Case

The bank is seeded with size-1 expressions:

- **Integer hole** $\square_{\mathbb{Z}}$ of type int (see Integer Holes below)
- **Boolean constants** $\top, \bot$ of type bool
- **Empty lists** $[\,]$ of type list[σ] for each list type σ
- **Input variable** x : list[int] (since all of our programs are of type list[int] → list[int], this is the variable that will be bound to the input list in a program like `length x`)

2.4 Integer Holes

Instead of seeding a fixed set of integer literals $C_{\text{seed}} = \{0, \ldots, 10\}$, the base case includes a single **integer hole** $\square_{\mathbb{Z}}$ of type int. A hole is an uninstantiated integer position; its concrete value is determined by a **substitution** σ, an ordered list of integers indexed by pre-order traversal

position in the AST. Therefore, concrete programs are a tuple (π, σ) where π is a valid program sketch and σ is the substitution of concrete values for the integer holes. This allows us to enumerate / diversify programs involving integer constants much more efficiently.

Fingerprinting. To compute the fingerprint of a sketch π containing k holes, the compiler substitutes the i-th hole (in pre-order) with PROBE[i mod |PROBE|] where we arbitrarily chose the random sequence

$$\text{PROBE} = [5, 3, 7, 2, 9, 0, 8, 1, 6, 4].$$

The resulting fingerprint has length m (same as concrete programs). Two sketches that produce identical outputs under the probe substitution are considered observationally equivalent and deduplicated. False equivalences are possible but rare given the diversity of the test suite.

Corpus expansion. After enumeration, a sketch π can be expanded into concrete programs by sampling N_{fill} substitutions $\sigma \sim \text{Uniform}(C^k_{\text{seed}})$ and retaining those that pass the quality filter. This expansion is a standalone utility; the RL pipeline works directly on sketches.

2.5 First-Order Applications

For a first-order function $f : A_1 \times \cdots \times A_k \to R$ and target size s, the arguments must satisfy $\sum_{i=1}^{k} s_i = s - 1$ with each $s_i \geq 1$. For every such composition:

$$\mathcal{B}[R][s] \;\mathrel{+}=\; \big\{ f(a_1, \ldots, a_k) \;\big|\; a_i \in \mathcal{B}[A_i][s_i] \big\} \Big/ \sim$$

Polymorphic functions are pre-expanded into all valid monomorphic instantiations (as described in the Grammar section); each instantiation is then enumerated independently using the formula above.

2.6 Higher-Order Applications and Nested Lambdas

When an argument position requires a function type $A_1 \times \cdots \times A_k \to B$, the lambda argument is synthesised inline rather than looked up in the flat bank. Given a size budget b for the lambda, an extended typing context $\Gamma' = \Gamma \cup \{y_i : A_i\}_{i=1}^{k}$ is formed, and a child bank $\mathcal{B}'$ is enumerated recursively over Γ' up to size $b - 1$. The lambda candidates are then:

$$\big\{ (\lambda\ (y_1 \cdots y_k)\ e) \;\big|\; e \in \mathcal{B}'[B][b-1] \big\}$$

Child banks are keyed by (Γ', b) and cached so that the same lambda type appearing as an argument to several functions at the same size is only built once.

Nesting depth. ℓ counts the number of enclosing lambda scopes. Higher-order calls within a child context are only attempted when $\ell \leq \ell_{\max}$ (default $\ell_{\max} = 2$), preventing combinatorial explosion from arbitrarily deep lambda nesting while still enabling patterns with three levels of nesting such as map ($\lambda y.$ map ($\lambda z.$ map ($\lambda w.\ \cdots$) z) y) x.

2.7 Quality Filtering

After enumeration, the corpus is obtained by retaining programs that pass three conditions. Let $\mathcal{F}(p) \subseteq \{1, \ldots, m\}$ denote the indices on which p does not crash. A program p is retained iff:

1. $|\mathcal{F}(p)| \geq 3$ (non-crashing on sufficiently many inputs)
2. $|\{\, p(t_i) \mid i \in \mathcal{F}(p)\}| \geq 2$ (non-constant output)
3. $\dfrac{|\{\, p(t_i) \mid i \in \mathcal{F}(p)\}|}{|\mathcal{F}(p)|} \geq \nu_{\min}$ (output variability, default $\nu_{\min} = 0.3$)

2.8 Key Parameters

Parameter	Default	Role
$s_{\max}$	8	Maximum AST size enumerated
$\ell_{\max}$	2	Maximum lambda nesting depth for HOF arguments
N_{fill}	10	Random substitutions sampled per sketch during corpus expansion
$\nu_{\min}$	0.3	Corpus variability threshold
T	10 inputs	Test suite for fingerprinting and deduplication

3 Part 2: Reinforcement Learning

The RL phase trains a neural policy on the enumerated corpus to generate good programs recursively from the root, making one decision per AST node. This is framed as a Markov Decision Process (MDP) over a grammar-constrained action space.

3.1 MDP Formulation

State. A synthesis state s is a tuple $(\tau, \Gamma, f_{\text{par}}, i, \delta, \ell)$:

Component	Description
τ	Required type at the current node
Γ	Typing context: variables in scope with their types
$f_{\text{par}},\ i$	Parent function and argument index (contextual features)
δ	Remaining depth budget (decremented at each recursive call)
ℓ	Current lambda nesting depth

Actions. The valid action set $\mathcal{A}(s)$ depends on s:

Action	Condition
Integer hole $\square_{\mathbb{Z}}$	$\tau = \text{int}$
Literal boolean	$\tau = \text{bool}$
Empty list	τ is a list type
Variable y	$y \in \text{dom}(\Gamma)$ with $\Gamma(y) = \tau$
Apply f (instance)	$\delta > 0$; return type of f matches τ; if f is higher-order, $\ell < \ell_{\max}^{\text{RL}} = 2$
Lambda	τ is a function type
If	$\delta > 0$

Episode. Generation begins from the state $s_0 = (\text{list[int]} \to \text{list[int]},\ \varnothing,\ \cdot,\ \cdot,\ \delta_{\max},\ 0)$ and proceeds by recursively expanding each chosen action into child states:

- Apply $f(A_1, \ldots, A_k) \to R$: recurse on $(\tau \leftarrow A_i,\ \delta - 1)$ for each i
- Lambda $(\tau_1 \times \cdots \times \tau_k \to \sigma)$: recurse on $(\tau \leftarrow \sigma,\ \Gamma \cup \{y_i : \tau_i\},\ \delta - 1,\ \ell + 1)$
- If: recurse on (bool, $\delta - 1$) then twice on $(\tau,\ \delta - 1)$

If any recursive call encounters an empty action set, the episode fails and returns no program.

3.2 Reward

Given a completed program p, let $\Phi(p)$ denote its fingerprint on T, and let $\mathcal{K}$ be the set of fingerprints already known (from the enumeration corpus and prior RL discoveries). The reward is:

$$R(p) = \mathrm{var}(\Phi(p)) \times \begin{cases} 1.0 & \text{if } \Phi(p) \notin \mathcal{K} \\ 0.1 & \text{otherwise} \end{cases}$$

where $\mathrm{var}(\Phi(p)) = |\{p(t_i)\}|/m$ is the fraction of distinct outputs. Programs with $|\mathcal{F}(p)| < 3$ or $|\{p(t_i)\}| < 2$ receive $R(p) = 0$ and are discarded.

The novelty multiplier incentivises exploration of behaviours not yet covered, while the 0.1 floor ensures the policy is not penalised for rediscovering known programs.

3.3 Priority Queue Buffer

The buffer follows the priority queue training scheme of Abolafia et al. (2018). $\mathcal{B}_{\text{buf}}$ maintains the top-K programs by reward (default $K = 5000$). It is a bounded min-heap: a new program p is admitted only if $R(p) > R_{\min}(\mathcal{B}_{\text{buf}})$, evicting the current minimum. Duplicate fingerprints are excluded.

Training samples are drawn uniformly from $\mathcal{B}_{\text{buf}}$ with no recency or priority weighting.

3.4 Policy Network

The policy π_θ maps a state s to a distribution over actions via a masked softmax. A network g_θ produces a logit for each action; invalid actions are excluded by setting their logits to $-\infty$ before normalising:

$$\pi_\theta(a \mid s) = \frac{\exp\big(g_\theta(s)_a\big)}{\sum_{a' \in \mathcal{A}(s)} \exp\big(g_\theta(s)_{a'}\big)}, \qquad a \in \mathcal{A}(s)$$

where $g_\theta : \mathcal{S} \to \mathbb{R}^{|\mathcal{A}|}$ is a two-layer feedforward network:

$$g_\theta(s) = W_2\, \mathrm{ReLU}(W_1\, \mathrm{enc}(s) + b_1) + b_2$$

State encoder. Six features of s are each embedded or projected into $\mathbb{R}^d$, then concatenated and projected back to $\mathbb{R}^d$:

Feature	Encoding
τ	Learned embedding over a type vocabulary
f_{par}	Learned embedding over the function vocabulary
i (arg index)	Learned embedding, clamped to $[0, 7]$
δ	Learned embedding, clamped to $[0, 15]$
ℓ	Learned embedding, clamped to $[0, 3]$
Γ	Linear projection of a per-type variable-count vector

Default $d = 64$, hidden dimension 128. Invalid actions are masked to $-\infty$ before softmax; the valid set $\mathcal{A}(s)$ is cached by $(\tau, \Gamma, \delta > 0, \ell)$.

3.5 Training Phase 1: Behavioural Cloning Warm-Start

Each corpus program p induces a sequence of state–action pairs $\{(s_t, a_t)\}$ by a DFS traversal of its AST, recording at each node the state that a top-down policy would have been in and the action it would have taken. Aggregating over all corpus programs gives a dataset $\mathcal{D}$. The policy is pre-trained by minimising:

$$\mathcal{L}_{\text{BC}}(\theta) = -\mathbb{E}_{(s,\,a)\,\sim\,\mathcal{D}}\big[\log \pi_\theta(a \mid s)\big]$$

(50 epochs, batch size 64, Adam at $\eta = 10^{-3}$). Transitions involving actions outside the vocabulary are dropped.

3.6 Training Phase 2: RL Loop

Each iteration alternates between a sampling phase and a gradient phase.

Sampling (32 episodes per iteration): run an episode under π_θ to obtain a sketch π (which may contain integer holes $\square_{\mathbb{Z}}$); sample $N_{\text{fill}} = 10$ random substitutions $\sigma \sim \text{Uniform}(C_{\text{seed}}^k)$ where k is the number of holes in π; select the σ^* maximising R; compute $\Phi(\pi[\sigma^*])$ and $R(\pi[\sigma^*])$; if $R > 0$, attempt insertion of the concrete program into $\mathcal{B}_{\text{buf}}$; if Φ is novel, add it to $\mathcal{K}$.

Gradient update (8 steps per iteration): sample a batch from $\mathcal{B}_{\text{buf}}$ and minimise the reward-weighted log-likelihood loss (Abolafia et al., 2018):

$$\mathcal{L}_{\text{RL}}(\theta) = -\mathbb{E}_{p\,\sim\,\mathcal{B}_{\text{buf}}}\left[R(p)\sum_t \log \pi_\theta(a_t \mid s_t)\right]$$

No baseline is subtracted. Gradient norms are clipped to 1.0 before the Adam step ($\eta = 10^{-4}$).

3.7 Trajectory Extraction

The dataset $\mathcal{D}$ (used in both BC and RL) is constructed by a DFS that is the inverse of the episode procedure. Given a complete AST, each node yields a state–action pair:

- λ-node $\rightarrow$ action **Lambda**, recurse into body at $\ell + 1$
- Application $f(a_1, \ldots, a_k) \rightarrow$ action **Apply** f, recurse into each a_i with $f_{\text{par}} = f$, i set accordingly

- If-node → action **If**, recurse into condition then branches
- Leaf → the appropriate literal or variable action

For polymorphic functions, the type instantiation is recovered by matching the inferred return type of f against τ in the current state.

4 Overall Pipeline

This section describes the procedure used to generate 5,699,938 unique and behaviourally diverse programs using the list manipulation DSL primitives provided in Rule et al. (2024), linking all of the above methods.

- **Phase 1 — Bottom-Up Enumeration** (cf. §2). A `BottomUpEnumerator` is run with parameters $s_{\max} = 8$, $\ell_{\max} = 2$, and $\nu_{\min} = 0.3$ on the default 10-input test suite and seed constants $C_{\text{seed}} = \{0, 1, \ldots, 10\}$. The enumerator fills the program bank $\mathcal{B}$ with all behaviourally distinct *sketches* (ASTs that may contain integer holes $\square_{\mathbb{Z}}$) up to size $s_{\max}$, yielding 643,837 sketches in total. Two sub-products are retained:
  - `quality_corpus` (581,984 sketches) — those that pass the quality filter (§2.7): at least 3 non-crashing inputs, at least 2 distinct outputs, and output variability $\geq \nu_{\min}$. Used for warm-start training and buffer seeding.
  - `all_sketches` (643,837 sketches) — the full bank regardless of quality, flattened across all types and sizes. Used as the input to sketch expansion in Phase 2.

  Quality sketches are serialised to `enum_sketches.json`; the full enumeration result is checkpointed to `enum_corpus.pkl` so that Phases 1 and 2 can be skipped on subsequent runs.
- **Phase 2 — Sketch Expansion** (cf. §2.4). Every sketch in `all_sketches` is concretised by the `expand_sketches` utility. For a sketch π containing k holes, substitutions $\sigma \in C_{\text{seed}}^k$ are enumerated exhaustively when $|C_{\text{seed}}|^k$ is small, or otherwise sampled uniformly, up to a per-sketch budget derived from

$$N_{\text{sub}} = \left\lceil \frac{N_{\text{target}}}{|\texttt{all\_sketches}|} \right\rceil, \qquad N_{\text{target}} = 6{,}000{,}000.$$

  Each concrete program is fingerprinted (§2.2), checked against a running deduplication table, and retained only if it passes the quality filter. In practice, most sketches contain zero integer holes and therefore contribute exactly one concrete program; the resulting deduplicated set `enum_concrete` contains 1,793,723 programs, written to `enum_corpus.json`. All fingerprints are added to `corpus_fingerprints`, which tracks every behaviour seen so far.
- **Phase 3 — RL Collection and Expansion** (cf. §3).
  - *Warm-start* (§3.5). The policy network is initialised via behavioural cloning on `quality_corpus`, subsampled to 100,000 programs (yielding 879,141 state–action transitions). Training runs for 5 epochs with batch size 128 and Adam at $\eta = 10^{-3}$, using GPU acceleration when available (MPS on Apple Silicon, CUDA otherwise). The trained weights are saved to `policy_warmstart.pt` so that warm-start can be skipped on subsequent runs.

– *Buffer seeding* (§3.3). The priority queue buffer (capacity $K = 50{,}000$) is seeded with programs from `quality_corpus` whose trajectories can be extracted; each is inserted with reward computed against `corpus_fingerprints`. In this run, the buffer was filled to capacity (50,000 programs).

– *RL loop* (§3.6). Up to 5,000,000 iterations are run, stopping early once 100,000 novel sketches have been collected. Each iteration consists of:

1. *Sampling* (32 episodes): an `Episode` is rolled out under the current policy with MDP depth budget $\delta_{\max} = 12$; the resulting sketch π is filled with up to $N_{\text{fill}} = 4$ random substitutions via `_fill_in_sketch`; the substitution maximising R is kept; the program is inserted into the buffer and, if its fingerprint is novel, appended to `rl_sketches` and added to `corpus_fingerprints`.
2. *Training* (8 gradient steps of batch size 64): a batch is drawn uniformly from the buffer and the reward-weighted log-likelihood loss $\mathcal{L}_{\text{RL}}$ is minimised with Adam ($\eta = 10^{-4}$) and gradient clipping to 1.0.

Checkpoints of `rl_sketches` are written every 10,000 iterations.

– *Post-RL expansion*. The 100,000 novel RL sketches are expanded by `expand_sketches` with `max_substitutions` = 100 per sketch and `target_total` = 4,000,000. The expanded concrete programs are deduplicated against all fingerprints in `corpus_fingerprints` (i.e. against the entire enumeration corpus and all RL-sampled programs); only genuinely novel behaviours are retained. This yields 3,906,215 novel programs, written to `rl_corpus.json`.

- **Final corpus.** The two output files together form the full dataset:

$$\underbrace{1{,}793{,}723}_{\texttt{enum\_corpus}} + \underbrace{3{,}906{,}215}_{\texttt{rl\_corpus}} = 5{,}699{,}938 \text{ programs.}$$

Program counts per stage.

Stage	Output file	Sketches	Concrete programs
Phase 1: Enumeration	`enum_sketches.json`	643,837	—
Phase 1: Quality filter	(in-memory)	581,984	—
Phase 2: Sketch expansion	`enum_corpus.json`	—	1,793,723
Phase 3: RL sampling	(in-memory)	100,000	—
Phase 3: Post-RL expansion	`rl_corpus.json`	—	3,906,215
Total			**5,699,938**

A Corpus Analysis

This section characterises the generated corpus along three axes: structural complexity, type and primitive coverage, and the diversity of the enumeration and RL sub-corpora.

A.1 Structural Complexity

The two sub-corpora occupy largely disjoint regions of the program-size spectrum. Because the enumerator is bounded by $s_{\max} = 8$, its output is concentrated at the upper end of that range: 83.9% of enumeration programs have size exactly 8, with a median of 8 and mean of 7.8. The RL sub-corpus, generated with MDP depth budget $\delta_{\max} = 12$, covers a much broader range. RL

program sizes span 5–98, with a median of 22 and mean of 23.9. The bulk of the RL corpus (43.2%) falls in the size 21–35 range, while a long tail extends to programs of size 98. Table 1 summarises the distribution.

Table 1: Program size distribution across the two sub-corpora.

	Min	Median	Mean	Max
Enumeration ($n = 1{,}793{,}723$)	1	8	7.8	8
RL ($n = 3{,}906{,}215$)	5	22	23.9	98

The RL size distribution is approximately bell-shaped (Table 2), indicating that the policy learns to produce programs of moderate depth rather than degenerately short or excessively long ones.

Table 2: RL corpus breakdown by size bucket.

Size range	Programs	Fraction
5–8	75,376	1.9%
9–12	594,142	15.2%
13–20	980,010	25.1%
21–35	1,687,452	43.2%
36–50	479,966	12.3%
51–98	89,269	2.3%

A.2 Type and Primitive Coverage

Because the MDP generates programs of type list[int] → list[int], the RL sub-corpus is homogeneous in its return type. Return-type diversity comes entirely from the enumeration sub-corpus, which covers all five concrete types: list[int] (51.5%), list[list[int]] (23.0%), int (22.9%), list[bool] (2.5%), and bool (0.1%).

Primitive coverage is near-complete in both sub-corpora: enumeration programs use 48 of the grammar's primitives and RL programs use 47, with the union covering all 48. The two sub-corpora differ in *how* primitives are combined. Higher-order functions are substantially more prevalent in the RL corpus: lambda expressions appear 29× more frequently per program in RL than in enumeration, and 3.3% of RL programs (128,414 programs) contain nested lambdas (i.e. reference inner parameters `_p1` or `_p2`), compared to a negligible fraction of enumeration programs. Multi-argument list operations—`splice`, `insert`, `concat`, `cut_slice`, `takelast`—are 5–8× more prevalent in the RL corpus, as these operations require larger programs to use in compositionally interesting ways.

A.3 Diversity

All programs have distinct fingerprints and all pass pass the quality filter (variability ≥ 0.3, non-crashing, non-constant). The enumeration corpus covers programs of size 1-8, while the RL helps extend beyond this size limit.

Some RL-generated programs suffer from internal redundancies. While their overall behaviour is interesting, they may be represented by shorter programs. For example, `product (repeat 10 8)` is an constant internal node in the AST. In the future, it may be worth simplifying RL-generated programs.

References

Daniel A Abolafia, Mohammad Norouzi, Jonathan Shen, Rui Zhao, and Quoc V Le. Neural program synthesis with priority queue training. *arXiv preprint arXiv:1801.03526*, 2018. *In sec. 2, 3.3, and 3.6.*

Joshua S Rule, Steven T Piantadosi, Andrew Cropper, Kevin Ellis, Maxwell Nye, and Joshua B Tenenbaum. Symbolic metaprogram search improves learning efficiency and explains rule learning in humans. *Nature Communications*, 15(1):6847, 2024. *In sec. 4.*